UNIVERSIDADE FEDERAL DE RORAIMA - UFRR
PROGRAMA DE PÓS GRADUAÇÃO EM COMPUTAÇÃO APLICADA - PPCA
MESTRADO PROFISSIONAL EM INFORMÁTICA NA EDUCAÇÃO

Quick Sort

Janderson Gomes

Estrutura de Dados | Prof. Dr. Filipe Dwan Pereira
Novembro de 2026

Roteiro

1. Contexto histórico
2. O algoritmo
3. Passo a passo com 6 números
4. Variações e versões modernas
5. Análise de complexidade
6. Resultados experimentais
7. Comparações
8. Usos, vantagens, limitações

Legenda nas figuras



pivô · menores que o pivô · elemento em exame · geral.

1

Contexto histórico

O problema da ordenação

Dada uma sequência de n registros $\langle a_1, a_2, \ldots, a_n \rangle$, ordenar é encontrar uma permutação $\langle a'_1, a'_2, \ldots, a'_n \rangle$ tal que

$$a'_1 \leq a'_2 \leq \cdots \leq a'_n$$

para uma relação de ordem sobre as chaves [Knuth, 1998, Cormen et al., 2022].

- **Por quê?** Ordenar acelera a busca (busca binária, $O(\log n)$), a detecção de duplicatas e a intercalação de conjuntos.
- **Modelo de custo:** comparações entre chaves e movimentações de registros [Knuth, 1998].
- **Limite inferior:** qualquer ordenação por comparações exige $\Omega(n \log n)$ comparações no pior caso [Cormen et al., 2022].

Propriedades que usaremos

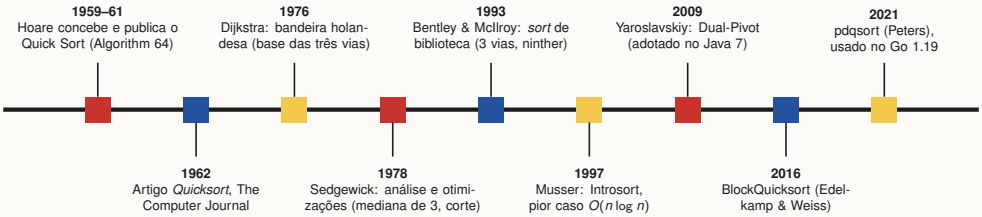
In-place: usa $O(1)$ ou $O(\log n)$ de memória extra.
Estável: preserva a ordem relativa de chaves iguais.
Adaptativo: aproveita ordem parcial já existente.

Tony Hoare e a invenção do Quick Sort



- **C. A. R. (Tony) Hoare** (n. 1934), cientista da computação britânico; Prêmio Turing de 1980 por contribuições ao projeto e à definição de linguagens de programação.
- **1959–60:** estudante visitante na Universidade Estatal de Moscou, trabalhando com tradução automática; precisou ordenar palavras para consulta a um dicionário e concebeu o Quick Sort [Shustek, 2009].
- **1961:** publicado como *Algorithm 64: Quicksort* (*Communications of the ACM*) [Hoare, 1961a]; **1962:** artigo completo com análise, no *The Computer Journal* [Hoare, 1962].
- Na mesma época, *Algorithm 65: Find* (hoje *quickselect*) usa a mesma partição para achar o k-ésimo menor elemento [Hoare, 1961b].

Linha do tempo



Fontes: [Hoare, 1961a, Hoare, 1962, Dijkstra, 1976, Sedgewick, 1978, Bentley e McIlroy, 1993, Musser, 1997, Wild e Nebel, 2012, Edelkamp e Weiss, 2016, Peters, 2021].

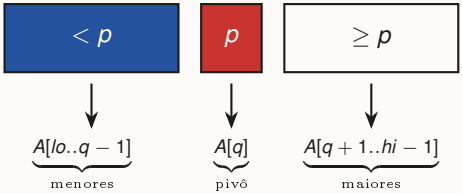
2

O algoritmo

Ideia central: dividir e conquistar

1. **Escolher** um elemento, o *pivô*.
2. **Particionar** o vetor: elementos menores que o pivô à esquerda, os demais à direita; o pivô fica em sua posição definitiva.
3. **Recursão** nas duas partes.

Não há etapa de “combinação”: ao final das chamadas, o vetor já está ordenado [Cormen et al., 2022].



Vetor após a partição.

QuickSort: pseudocódigo e C

Pseudocódigo

```
1 QUICKSORT(A, lo, hi)
2 1  se lo < hi então
3 2      p ← PARTICIONAR(A, lo, hi)
4 3      QUICKSORT(A, lo, p - 1)
5 4      QUICKSORT(A, p + 1, hi)
```

Primeira chamada

`QUICKSORT(A, 0, n - 1)`, com índices inclusivos e base 0.

Linguagem C

```c
void quicksort(int A[], int lo, int hi) {
    if (lo < hi) {
        int p = particionar(A, lo, hi);
        quicksort(A, lo, p - 1);
        quicksort(A, p + 1, hi);
    }
}
```

Partição de Lomuto (pivô = último elemento)

Pseudocódigo

```
1  PARTICIONAR–LOMUTO(A, lo, hi)
2  1  pivô ← A[hi]
3  2  s ← lo        ▷ A[lo..s-1] < pivô
4  3  para i ← lo até hi - 1 faça
5  4     se A[i] < pivô então
6  5        trocar A[i] ↔ A[s]
7  6        s ← s + 1
8  7  trocar A[s] ↔ A[hi]
9  8  retorne s
```

Linguagem C

```c
static void troca(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

static int particionar(int A[],
                       int lo, int hi) {
    int pivo = A[hi];
    int s = lo;       /* A[lo..s-1] < pivo */
    for (int i = lo; i < hi; i++)
        if (A[i] < pivo)
            troca(&A[i], &A[s++]);
    troca(&A[s], &A[hi]);
    return s;
}
```

Esquema atribuído a Nico Lomuto e popularizado por Bentley [Bentley, 2000] e por [Cormen et al., 2022]. No trabalho final, optou-se por utilizar essa forma, isto é, uma versão com intervalo semiaberto.

Partição de Hoare (dois ponteiros, pivô = primeiro)

Pseudocódigo

```
1  PARTICIONAR–HOARE(A, lo, hi)
2  1  pivô ← A[lo]
3  2  i ← lo - 1;  j ← hi + 1
4  3  enquanto verdadeiro faça
5  4      repita i ← i + 1 até A[i] ≥ pivô
6  5      repita j ← j - 1 até A[j] ≤ pivô
7  6      se i ≥ j então retorne j
8  7      trocar A[i] ↔ A[j]
```

Recursão em `(lo, p)` e `(p+1, hi)`; o pivô *não* fica necessariamente em $A[p]$ [Cormen et al., 2022].

Linguagem C

```c
static int part_hoare(int A[],
                      int lo, int hi) {
    int pivo = A[lo];
    int i = lo - 1, j = hi + 1;
    while (1) {
        do { i++; } while (A[i] < pivo);
        do { j--; } while (A[j] > pivo);
        if (i >= j) return j;
        troca(&A[i], &A[j]);
    }
}

void qs_hoare(int A[], int lo, int hi) {
    if (lo < hi) {
        int p = part_hoare(A, lo, hi);
        qs_hoare(A, lo, p);
        qs_hoare(A, p + 1, hi);
    }
}
```

Lomuto $\times$ Hoare

	Lomuto	**Hoare (original)**
Varredura	Um ponteiro, da esquerda para a direita.	Dois ponteiros que se aproximam.
Pivô na partição	Termina na posição final.	Fica em algum ponto da fronteira.
Trocas (média)	Mais numerosas (ordem de $n \ln n$ no total)	Cerca de 1/3 das de Lomuto. [Knuth, 1998, Sedgewick, 1978]
Entradas ordenadas.	$\Theta(n^2)$ com pivô fixo nas pontas.	Idem com pivô fixo nas pontas.
Chaves repetidas.	Degrada com muitos iguais (usa $<$ ou $\leq$).	Divide melhor, pois ambos os ponteiros param nos iguais.
Simplicidade	Muito simples; invariante direta.	Mais delicada (limites dos índices).

Sobre o invariante da partição de Lomuto

Invariante (início de cada iteração, índice i)

- $lo \leq k < s \Rightarrow A[k] < p$
- $s \leq k < i \Rightarrow A[k] \geq p$
- $A[hi]$ é o elemento em exame
- $A[hi] = p$

Inicialização: $i = s = lo$, regiões vazias.
Manutenção: se $A[i] < p$, a troca com $A[s]$ estende a região azul; senão, estende a branca.
Término: com $i = hi$, trocar $A[s] \leftrightarrow A[hi]$ deixa p entre as duas regiões.

Correção do QuickSort (indução em n)

Base: $n \leq 1$ já está ordenado.

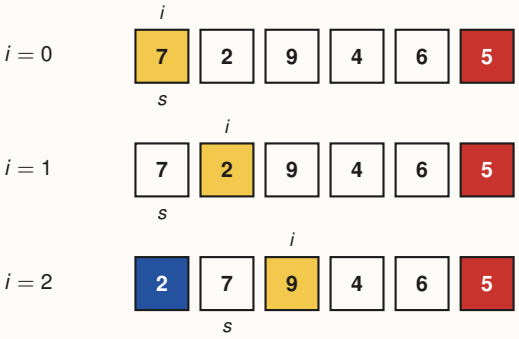
Passo: após a partição, todo elemento à esquerda é $< p$ e todo à direita é $\geq p$. Por hipótese, as duas partes (menores que n) são ordenadas pelas chamadas recursivas; logo o vetor inteiro fica ordenado.

Terminação: cada chamada recebe um subvetor estritamente menor, pois o pivô sai das partes.

3

Passo a passo com 6 números

Entrada $A = [7, 2, 9, 4, 6, 5]$: partição 1 (1/3)

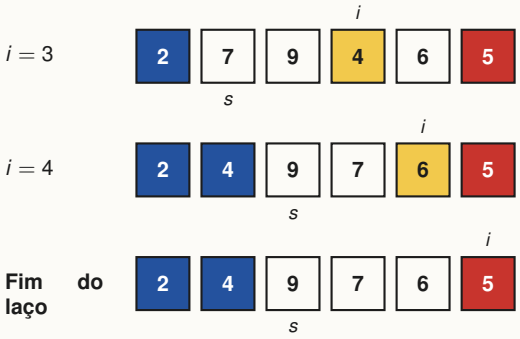


Pivô $= A[5] = 5$ (último), $s = 0$. $7 < 5$**?** **Não.** Nada muda.

$2 < 5$**? Sim.** Trocar $A[1] \leftrightarrow A[0]$ e $s \leftarrow 1$.

$9 < 5$**? Não.** O 2 já ocupa a região azul ($A[0]$).

Partição 1 (2/3)



$4 < 5$**? Sim.** Trocar $A[3] \leftrightarrow A[1]$ (o 7 vai para a posição 3) e $s \leftarrow 2$.

$6 < 5$**? Não.**

$i = hi$. Falta posicionar o pivô: trocar $A[s] = A[2] \leftrightarrow A[5]$.

Partição 1 (3/3): resultado

Depois da troca final:



O pivô **5** está na posição definitiva (índice 2).
À esquerda, tudo < 5; à direita, tudo ≥ 5.

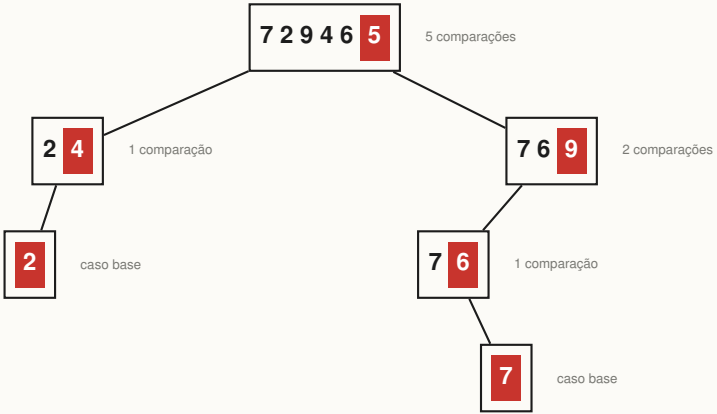
Custo: 5 comparações ($n - 1$) e 3 trocas.

Subproblemas: $A[0..1] = [2, 4]$ e $A[3..5] = [7, 6, 9]$.

Leitura do invariante

Após cada passo, a região azul contém somente valores menores que o pivô; os valores maiores ficam para trás, “empurrados” pelas trocas. Cada elemento é comparado com o pivô **uma única vez** nesta partição.

Recursão: árvore de chamadas



Total: $5 + 1 + 2 + 1 = \mathbf{9}$ comparações. Esperado para $n = 6$ em média: $2(n+1)H_n - 4n \approx 10{,}3$.

Rastreio das chamadas

Chamada	Subvetor	Pivô	Após a partição	Comp.
QS(0,5)	[7, 2, 9, 4, 6, 5]	5	[2, 4] **5** [7, 6, 9]	5
QS(0,1)	[2, 4]	4	[2] **4** [ ]	1
QS(3,5)	[7, 6, 9]	9	[7, 6] **9** [ ]	2
QS(3,4)	[7, 6]	6	[ ] **6** [7]	1
Casos base (tamanho ≤ 1): QS(0,0), QS(2,1), QS(3,2), QS(4,4), ...				0
Total de comparações				**9**

Resultado final:



■ = pivôs posicionados; □ = elementos resolvidos pelo caso base.

4

Variações e versões modernas

Escolha do pivô

Estratégia	Como funciona	Efeito
Primeiro/último	usa $A[lo]$ ou $A[hi]$	simples; $\Theta(n^2)$ em entradas ordenadas
Aleatório	sorteia o pivô em $[lo, hi]$	pior caso improvável para qualquer entrada [Cormen et al., 2022]
Mediana de três	mediana de $A[lo]$, $A[meio]$, $A[hi]$	trata bem entradas ordenadas; menos comparações [Sedgewick, 1978]
Ninther	mediana de três medianas de três (vetores grandes)	pivô mais próximo da mediana [Bentley e McIlroy, 1993]

Melhorias práticas em C

Mediana de três (pivô vai para A[hi])

```c
static void mediana3(int A[],
                     int lo, int hi) {
    int mid = lo + (hi - lo) / 2;
    if (A[mid] < A[lo])
        troca(&A[mid], &A[lo]);
    if (A[hi] < A[lo])
        troca(&A[hi], &A[lo]);
    if (A[hi] < A[mid])
        troca(&A[hi], &A[mid]);
    /* a mediana vai para A[hi] */
    troca(&A[mid], &A[hi]);
}
```

Pivô aleatório: antes de particionar, trocar $A[hi]$ com $A[lo + \texttt{rand()} \bmod (hi - lo + 1)]$.

Pilha $O(\log n)$: recursão na menor parte

```c
void qs_pilha(int A[], int lo, int hi) {
    while (lo < hi) {
        mediana3(A, lo, hi);
        int p = particionar(A, lo, hi);
        if (p - lo < hi - p) {
            qs_pilha(A, lo, p - 1);
            lo = p + 1;
        } else {
            qs_pilha(A, p + 1, hi);
            hi = p - 1;
        }
    }
}
```

Recursar sempre na menor parte limita a profundidade a $\lfloor \log_2 n \rfloor$

[Sedgewick, 1978, Cormen et al., 2022].

Partição em três vias (Dijkstra)

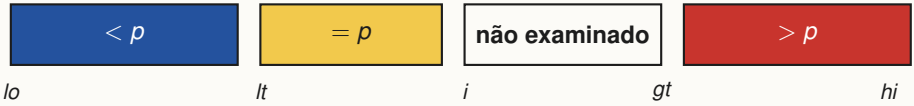
Pseudocódigo

```
1  QUICKSORT–3VIAS(A, lo, hi)
2  1  se lo ≥ hi então retorne
3  2  pivô ← A[lo];  lt ← lo
4  3  i ← lo + 1;  gt ← hi
5  4  enquanto i ≤ gt faça
6  5     se A[i] < pivô então
7  6        trocar A[lt] ↔ A[i]; lt++; i++
8  7     senão se A[i] > pivô então
9  8        trocar A[i] ↔ A[gt]; gt--
10 9     senão i++
11 10 QUICKSORT–3VIAS(A, lo, lt - 1)
12 11 QUICKSORT–3VIAS(A, gt + 1, hi)
```

Linguagem C

```c
void qs3(int A[], int lo, int hi) {
    if (lo >= hi) return;
    int pivo = A[lo];
    int lt = lo, i = lo + 1, gt = hi;
    while (i <= gt) {
        if (A[i] < pivo)
            troca(&A[lt++], &A[i++]);
        else if (A[i] > pivo)
            troca(&A[i], &A[gt--]);
        else
            i++;
    }
    qs3(A, lo, lt - 1);
    qs3(A, gt + 1, hi);
}
```

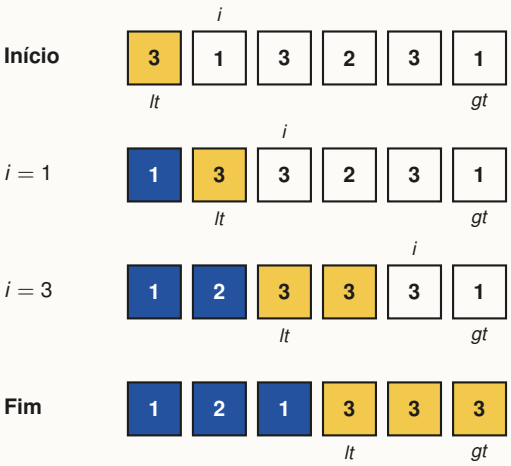
Invariante das três vias



- $A[lo..lt-1] < p$ (est. menores que p)
- $A[lt..i-1] = p$ (iguais a p)
- $A[i..gt]$ (ainda por examinar)
- $A[gt+1..hi] > p$ (est. maiores que p)

A região dos iguais ao pivô nunca mais é revisitada; a recursão ocorre só nas regiões azul e vermelha [Dijkstra, 1976, Sedgewick e Wayne, 2011]. É a solução do *problema da bandeira holandesa*, de Dijkstra [Dijkstra, 1976].

Três vias: $A = [3, 1, 3, 2, 3, 1]$, pivô $= 3$



$lt = 0$, $i = 1$, $gt = 5$. O pivô ocupa a região amarela.

$1 < 3$: trocar $A[lt] \leftrightarrow A[i]$; lt, i avançam.

$2 < 3$: troca com $A[lt = 1]$ (o 3 do meio só avançou i).

$1 < 3$: troca com $A[lt = 2]$; $i > gt$. **Os três 3 ficam prontos**; resta ordenar $[1, 2, 1] \Rightarrow [1, 1, 2, 3, 3, 3]$.

Variantes modernas

Variante	Ano	Ideia	Onde aparece
Corte para inserção	1978	subvetores pequenos são ordenados por inserção [Sedgewick, 1978]	implementação popular
Introsort	1997	troca para Heap Sort ao exceder profundidade $\approx 2 \log_2 n$: pior caso $O(n \log n)$ [Musser, 1997]	`std::sort` em C++ (libstdc++)
Dual-pivot	2009	dois pivôs, três partes; menos trocas/cache misses [Wild e Nebel, 2012]	`Arrays.sort` (primitivos), Java 7+
BlockQuicksort	2016	partição em blocos evita erros de predição de desvio [Edelkamp e Weiss, 2016]	bibliotecas de alto desempenho
pdqsort	2021	detecta padrões e entradas ruins; reembaralha pivôs [Peters, 2021]	Go (`sort`, desde 1.19)

5

Análise de complexidade

Recorrências e resultados

Melhor caso (partição equilibrada)

$T(n) = 2\,T(n/2) + \Theta(n) \;\Rightarrow\; \Theta(n \log n)$

Pior caso (partição $n-1$ e 0)

$T(n) = T(n-1) + \Theta(n) \;\Rightarrow\; \Theta(n^2)$, com exatamente $\frac{n(n-1)}{2}$ comparações.
Ocorre, por exemplo, com pivô fixo em entrada já ordenada.

Caso médio (permutações equiprováveis)

$E[C_n] = 2(n+1)H_n - 4n \approx 2n \ln n \approx 1{,}39\, n \log_2 n$
[Knuth, 1998, Cormen et al., 2022]

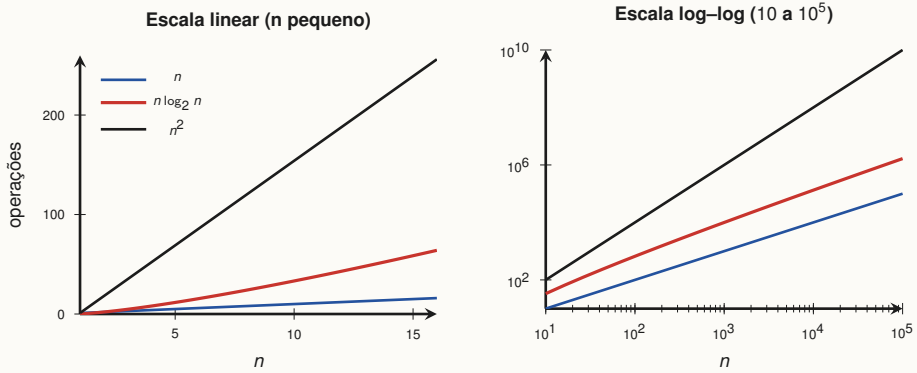
Memória auxiliar: só a pilha de recursão. Esperada $O(\log n)$; no pior caso $O(n)$, reduzida a $O(\log n)$ recursando na menor parte.

Estabilidade: *não* estável. Exemplo com Lomuto: $[2_a, 2_b, 1]$ com pivô 1 vira $[1, 2_b, 2_a]$; os dois 2 trocam de ordem.

Adaptatividade: não explora ordem prévia (com pivô fixo, a ordem prévia é *pior*).

Por que 1,39? A razão $2 \ln n / \log_2 n = 2 \ln 2 \approx 1{,}386$.

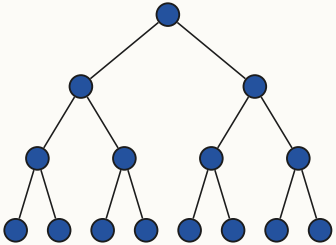
Gráficos teóricos: n, $n \log_2 n$ e n^2



Em $n = 10^5$: $n \log_2 n \approx 1{,}7 \times 10^6$ *versus* $n^2 = 10^{10}$, uma razão de quase 6 000 vezes.

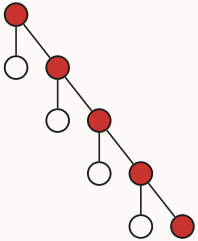
Árvores de recursão: melhor e pior caso

Melhor caso: profundidade $\approx \log_2 n$



cada nível custa $\Theta(n)$

Pior caso: profundidade n



níveis de custo $n,\ n-1,\ n-2, \ldots \Rightarrow \frac{n(n-1)}{2}$

6

Trabalho Final

Metodologia: `problems/lesson11`

- **Repositório:** `https://github.com/carauma7/art-mondrian` (C, console, Windows/MinGW).
- **Dados:** inteiros em [0, 99 999] gerados por um congruencial linear ($x \leftarrow 1664525\,x + 1013904223 \bmod 2^{32}$; valor $= x \bmod 10^5$), arquivos de **100, 1.000, 10.000 e 100.000** números.
- **Algoritmos:** Selection, Insertion, Bubble, Radix, Quick, Merge e Heap; cada um lê o arquivo, ordena e grava a saída em `data/<algoritmo>/`.
- **Medidas:** tempo por `QueryPerformanceCounter` (gravado no nome do arquivo, p. ex. `10000_961us.txt`), contagem de comparações e de movimentações.
- **Quick Sort implementado:** Lomuto com pivô no último elemento.



Metodologia: `main.exe`

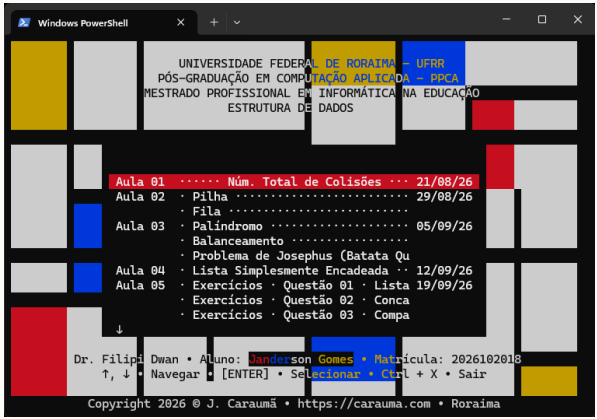


Figura: Aplicação desenvolvida em C como trabalho final.

Quick Sort implementado (`quick-sort.c`)

```c
static void quick_range(int *values, size_t left, size_t right,
                        Lesson11Metrics *metrics)
{
    size_t store = left, i;
    int pivot;
    if (right - left < 2) return;           /* caso base */
    pivot = values[right - 1];              /* pivô = último elemento */
    for (i = left; i + 1 < right; i++) {
        metrics->comparisons++;
        if (values[i] < pivot) {            /* troca i <-> store */
            int t = values[i]; values[i] = values[store]; values[store] = t;
            metrics->movements += 3;
            store++;
        }
    }
    /* troca values[store] <-> pivô (+3 movimentações) */
    quick_range(values, left, store, metrics);
    quick_range(values, store + 1, right, metrics);
}
```

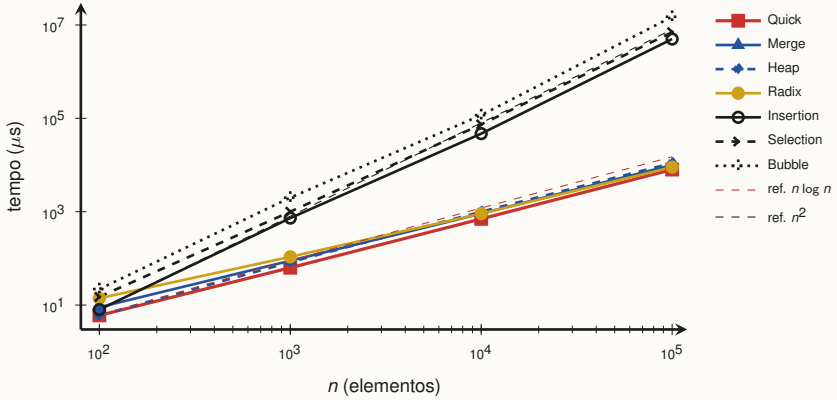
Trecho adaptado (contadores de progresso omitidos): é a partição de Lomuto, em intervalo semiaberto [*left*, *right*).

Tempos medidos (uma execução por tamanho)

Algoritmo	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Quick Sort	**6 µs**	**63 µs**	**701 µs**	**8 ms**
Merge Sort	9 µs	89 µs	911 µs	10 ms
Heap Sort	6 µs	83 µs	1 ms	11 ms
Radix Sort	14 µs	108 µs	911 µs	9 ms
Insertion Sort	8 µs	730 µs	47 ms	5 s
Selection Sort	15 µs	1 ms	74 ms	7 s
Bubble Sort	22 µs	2 ms	117 ms	15 s

Fonte: arquivos gerados pelo repositório (2026).
Tempos a partir de 1 ms aparecem arredondados no nome do arquivo; empates estão abaixo da resolução registrada.

Curvas obtidas $\times$ referências teóricas



Referências tracejadas ancoradas no ponto $n = 100$ (Quick: 6 µs; Insertion: 8 µs).

Crescimento observado × previsto

	100 → 1.000 (×)		1.000 → 10.000 (×)		10.000 → 100.000 (×)		
Algoritmo	observado	previsto	observado	previsto	observado	previsto	**Modelo**
Quick	10,5	15,0	11,1	13,3	11,4	12,5	$n \log n$
Merge	9,9	15,0	10,2	13,3	11,0	12,5	$n \log n$
Heap	13,8	15,0	12,0	13,3	11,0	12,5	$n \log n$
Radix	7,7	10	8,4	10	9,9	10	$d\,(n + b)$
Insertion	91	100	64	100	106	100	n^2
Selection	≈67	100	74	100	≈95	100	n^2
Bubble	≈91	100	≈59	100	≈128	100	n^2

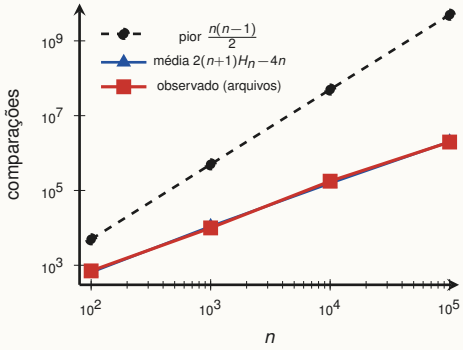
Previsto: $n \log_2 n$ cresce 15,0×, 13,3× e 12,5×; n^2 cresce 100×. Valores com "≈" dependem de tempos arredondados. O Quick Sort acompanha $n \log n$ [Cormen et al., 2022].

Comparações realizadas (execução do código do repositório)

Algoritmo	$n = 100$	$n = 1.000$	$n = 10.000$	$n = 100.000$
Quick Sort	**707**	**9.967**	**177.075**	**1.971.762**
Merge Sort	536	8.696	120.430	1.536.560
Heap Sort	1.042	16.844	235.352	3.019.814
Insertion Sort	2.554	252.542	24.960.251	*não medido*
Selection Sort	4.950	499.500	49.995.000	4.999.950.000
Bubble Sort	4.935	499.380	49.981.470	*não medido*
Radix Sort	sem comparações entre chaves (distribuição por dígitos)			

Contagens recalculadas sobre os arquivos atuais `inteiros_*.txt`; Insertion e Bubble em $n = 100.000$ foram omitidos por serem quadráticos e levarem vários segundos.

Quick Sort: contagem observada × teoria



n	obs.	teoria	desvio
100	707	648	+9%
1.000	9.967	10.986	−9%
10.000	177.075	155.772	+14%
100.000	1.971.762	2.018.053	−2%

Validação: 2.000 permutações aleatórias do mesmo algoritmo dão médias 647; 10.990; 155.825 (desvio-padrão 59; 639; 6.525), em acordo com a fórmula. O arquivo de $n = 10.000$ fica $\approx 3{,}4\sigma$ acima da média.

Hipótese (não testada): os bits baixos de um congruencial com módulo 2^{32} têm período curto [Knuth, 1997], e o valor é $x \bmod 10^5$; usar bits altos ou outro gerador deve aproximar o resultado da média.

7

Comparações com outros algoritmos

Tabela comparativa (teoria)

Algoritmo	Melhor	Médio	Pior	Memória extra	Estável	In-place
Selection	n^2	n^2	n^2	$O(1)$	não	sim
Insertion	n	n^2	n^2	$O(1)$	sim	sim
Bubble (com parada)	n	n^2	n^2	$O(1)$	sim	sim
Radix (LSD)	$d(n+b)$	$d(n+b)$	$d(n+b)$	$O(n+b)$	sim	não
Quick	$n \log n$	$n \log n$	n^2	$O(\log n)$ esp.; $O(n)$ pior	não	sim
Merge	$n \log n$	$n \log n$	$n \log n$	$O(n)$	sim	não
Heap	$n \log n$	$n \log n$	$n \log n$	$O(1)$	não	sim

Notação: $\Theta(\cdot)$ para tempo; d = dígitos, b = base. Valores conforme [Cormen et al., 2022, Knuth, 1998, Ziviani, 2010] e as anotações do próprio repositório.

Quick $\times$ Merge $\times$ Heap: quando preferir cada um

	Quick Sort	**Merge Sort**	**Heap Sort**
Pior caso	$\Theta(n^2)$ (evitável)	$\Theta(n \log n)$	$\Theta(n \log n)$
Memória extra	pilha $O(\log n)$	vetor auxiliar $O(n)$	$O(1)$
Estabilidade	não	sim	não
Acesso à memória	sequencial; boa localidade de cache	sequencial; exige cópias	saltos pela árvore; má localidade
Comparações (medido, $n = 10^5$)	1.971.762	1.536.560	3.019.814
Tempo (medido, $n = 10^5$)	8 ms	10 ms	11 ms
Indicado para	vetores em memória, uso geral	estabilidade; listas; dados externos	memória mínima e garantia de pior caso

O Merge Sort faz menos comparações, mas o Quick Sort compensa com menor custo por operação (sem cópias para vetor auxiliar), resultando em tempos equivalentes na escala medida [Sedgewick e Wayne, 2011].

8

Usos, vantagens, limitações

Vantagens e desvantagens

Vantagens

- Em média $\Theta(n \log n)$, com constante pequena (cerca de $1{,}39\, n \log_2 n$ comparações).
- *In-place*: sem vetor auxiliar; apenas pilha $O(\log n)$.
- Boa localidade de referência (varredura sequencial).
- Pouco código; fácil de adaptar (3 vias, *quickselect*, paralelismo).
- Base de implementações de bibliotecas [Bentley e McIlroy, 1993, Musser, 1997].

Desvantagens

- Pior caso $\Theta(n^2)$ com pivôs ruins; entradas adversárias existem [McIlroy, 1999].
- Não é estável.
- Recursão profunda no pior caso (risco de estouro de pilha).
- Sensível a muitos valores repetidos se a partição for de duas vias.
- Desempenho depende da escolha do pivô e de detalhes de implementação.

Por que usar o Quick Sort?

1. **Ordenação geral em memória:** é a escolha padrão quando não se exige estabilidade e se quer pouca memória extra.
2. **Desempenho prático:** constantes baixas e cache amigável; nos experimentos, o menor tempo em todos os tamanhos medidos (6 µs, 63 µs, 701 µs e 8 ms).
3. **Robustez com engenharia:** pivô aleatório/mediana de três, corte para inserção e Introsort eliminam o pior caso na prática [Sedgewick, 1978, Musser, 1997].
4. **Ideia reutilizável:** a mesma partição resolve seleção do k-ésimo elemento (*quickselect*) [Hoare, 1961b, Cormen et al., 2022].

Quando evitar

Quando é preciso **estabilidade** (usar Merge/Timsort), **pior caso garantido** sem engenharia adicional (Heap/Merge/Introsort) ou dados em **memória externa** (intercalação).

Principais usos na literatura

Contexto	Uso	Fonte
C++	`std::sort` é tipicamente um Introsort (Quick Sort + Heap Sort + inserção)	[Musser, 1997]
Java	`Arrays.sort` para tipos primitivos usa Dual-Pivot Quicksort	[Wild e Nebel, 2012]
Go	pacote `sort` usa pdqsort desde a versão 1.19	[Peters, 2021]
Seleção	mediana e k-ésimo menor em tempo esperado $O(n)$ (*quickselect*)	[Hoare, 1961b, Cormen et al., 2022]
Competições (ICPC, OBI, Maratona SBC)	usa-se o `sort` da biblioteca; adversários clássicos contra Quick Sort ingênuo motivam pivô aleatório e Introsort	[McIlroy, 1999]
Ensino	Knuth (§5.2.2), Cormen et al. (cap. 7), Sedgewick e Wayne (§2.3)	[Knuth, 1998, Cormen et al., 2022, Sedgewick e Wayne, 2011]

Onde estudar mais?

Internacionais

- **D. E. Knuth**, *The Art of Computer Programming*, vol. 3: análise detalhada do Quick Sort em §5.2.2 [Knuth, 1998].
- **Cormen, Leiserson, Rivest e Stein**: capítulo 7, partições de Lomuto e Hoare, análise e aleatorização [Cormen et al., 2022].
- **R. Sedgewick**: otimizações práticas [Sedgewick, 1978] e *Algorithms*, §2.3 [Sedgewick e Wayne, 2011].
- **J. Bentley**: *Programming Pearls* [Bentley, 2000].

Em português

- **N. Ziviani**, *Projeto de Algoritmos com Implementações em Pascal e C* [Ziviani, 2010].
- **Celes, Cerqueira e Rangel**, *Introdução a Estruturas de Dados com Técnicas de Programação em C* [Celes et al., 2017].
- **Szwarcfiter e Markenzon**, *Estruturas de Dados e seus Algoritmos* [Szwarcfiter e Markenzon, 2010].
- **P. Feofiloff**, *Algoritmos em Linguagem C* [Feofiloff, 2009].



Obrigado!

Viva o software livre!

© Caraumã (carauma.com) · Orgulhosamente feito em LaTeX

Referências I

[Bentley, 2000] J. Bentley. *Programming Pearls*. 2. ed. Addison-Wesley, 2000.

[Bentley e McIlroy, 1993] J. L. Bentley, M. D. McIlroy. Engineering a sort function. *Software: Practice and Experience*, 23(11):1249–1265, 1993.

[Celes et al., 2017] W. Celes, R. Cerqueira, J. L. Rangel. *Introdução a Estruturas de Dados com Técnicas de Programação em C*. 2. ed. Elsevier, 2017.

[Cormen et al., 2022] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*. 4. ed. MIT Press, 2022. (cap. 7, Quicksort)

[Dijkstra, 1976] E. W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.

[Edelkamp e Weiss, 2016] S. Edelkamp, A. Weiss. BlockQuicksort: how branch mispredictions don't affect quicksort. In: *ESA 2016*, LIPIcs 57, 38:1–38:16, 2016.

[Feofiloff, 2009] P. Feofiloff. *Algoritmos em Linguagem C*. Elsevier/Campus, 2009.

[Hoare, 1961a] C. A. R. Hoare. Algorithm 64: Quicksort. *Communications of the ACM*, 4(7):321, 1961.

[Hoare, 1961b] C. A. R. Hoare. Algorithm 65: Find. *Communications of the ACM*, 4(7):321–322, 1961.

[Hoare, 1962] C. A. R. Hoare. Quicksort. *The Computer Journal*, 5(1):10–16, 1962.

[Knuth, 1997] D. E. Knuth. *The Art of Computer Programming*, vol. 2: Seminumerical Algorithms. 3. ed. Addison-Wesley, 1997. (cap. 3, números aleatórios)

[Knuth, 1998] D. E. Knuth. *The Art of Computer Programming*, vol. 3: Sorting and Searching. 2. ed. Addison-Wesley, 1998. (§5.2.2)

[McIlroy, 1999] M. D. McIlroy. A killer adversary for quicksort. *Software: Practice and Experience*, 29(4):341–344, 1999.

[Musser, 1997] D. R. Musser. Introspective sorting and selection algorithms. *Software: Practice and Experience*, 27(8):983–993, 1997.

[Peters, 2021] O. R. L. Peters. Pattern-defeating quicksort. *arXiv*:2106.05123, 2021.

[Sedgewick, 1978] R. Sedgewick. Implementing Quicksort programs. *Communications of the ACM*, 21(10):847–857, 1978.

[Sedgewick e Wayne, 2011] R. Sedgewick, K. Wayne. *Algorithms*. 4. ed. Addison-Wesley, 2011. (§2.3)

[Shustek, 2009] L. Shustek. An interview with C.A.R. Hoare. *Communications of the ACM*, 52(3):38–41, 2009.

Referências II

[Szwarcfiter e Markenzon, 2010] J. L. Szwarcfiter, L. Markenzon. *Estruturas de Dados e seus Algoritmos*. 3. ed. LTC, 2010.

[Wild e Nebel, 2012] S. Wild, M. E. Nebel. Average case analysis of Java 7's dual pivot quicksort. In: *ESA 2012*, LNCS 7501, p. 825–836, Springer, 2012.

[Ziviani, 2010] N. Ziviani. *Projeto de Algoritmos com Implementações em Pascal e C*. 3. ed. Cengage Learning, 2010.